

Reviewer Pack — Ihara Zeta Entropy Lower-Bounds DPLL Depth on 3-Regular Tseitin

Entry 3a9492e9db22 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 19:45:50 UTC

Ihara Zeta Entropy Lower-Bounds DPLL Depth on 3-Regular Tseitin

Entry ID: 3a9492e9db22 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 19:45:50 UTC

1. Conjecture

Field A (mathematical branch): Ihara zeta functions of finite graphs (analytic number theory / spectral graph theory in the Stark-Terras-Storm tradition: closed primitive backtrackless cycle counting via the determinant identity $\zeta_G(u)^{-1} = (1-u^2)^{|E|-|V|} \cdot \det(I - uA + (d-1)u^2 I)$) **Field B** (complexity object): DPLL refutation depth (a polynomially-tight proxy for tree-like resolution depth, hence a lower bound on resolution width up to log factors via Ben-Sasson-Wigderson) of Tseitin formulas $T(G,c)$ on small 3-regular graphs G with odd total charge

Statement:

Define the normalized Ihara entropy of a 3-regular graph G at the regular point $u=1/3$ by $\eta(G) := (1/|V|) \cdot \sum_i \ln|1 - 3\lambda_i(A_G)|$, where $\{\lambda_i\}$ are the adjacency eigenvalues. Then for every connected 3-regular simple graph G on $|V| \leq 20$ vertices and every charge $c:V \rightarrow \{0,1\}$ with $\sum c(v) \equiv 1 \pmod{2}$, the DPLL refutation depth satisfies $d_{\text{DPLL}}(T(G,c)) \geq (1/4) \cdot |V| \cdot \eta(G)$, with the inequality strict whenever G is non-bipartite. A single (G,c) violating this bound falsifies the conjecture.

Rationale (proposer's reasoning):

The Ihara determinant aggregates spectral information about closed non-backtracking walks, the same combinatorial objects (short cycles in the underlying graph) that drive both expansion and the Ben-Sasson-Wigderson width lower bound for Tseitin; yet ζ_G has essentially never been used in proof complexity (the Terras school treats it as an analytic object). Because expander Tseitin formulas push $\eta(G)$ toward its 3-regular maximum (eigenvalues concentrated near 0 inflate $\sum \ln|1-3\lambda_i|$), an Ihara-driven lower bound would refine the BSW expansion bound by capturing higher-order non-backtracking structure invisible to the Cheeger/edge-expansion invariants already explored. The test does not relativize, evades natural proofs (η is highly non-large/non-constructive on random functions because it is a graph spectral statistic, not a function-of-truth-table property), and is algebrization-immune.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a0755e6f88e6ff95

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~ 600 instances ($|V| \in \{6, 8, 10, 12\}$, ≤ 30 graphs each, 5 odd-charges each), conjecture is SUPPORTED iff $\geq 95\%$ satisfy $d_DPLL \geq |V| \cdot \eta(G)/4$ AND every non-bipartite instance has strict inequality $d_DPLL > |V| \cdot \eta(G)/4$. FALSIFIED by any single (G, c) with $d_DPLL < |V| \cdot \eta(G)/4$, or any non-bipartite (G, c) with $d_DPLL = |V| \cdot \eta(G)/4$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Ihara zeta function Tseitin formula resolution - spectral graph expansion DPLL refutation depth Tseitin 3-regular - Ihara zeta entropy proof complexity resolution width

Top relevant hits considered: - [<http://arxiv.org/abs/1804.08448v8>] On Asymptotic Formula for the First Moment of the Riemann Zeta-Function on the Critical Line - [<http://arxiv.org/abs/math/0311029v1>] Zeta functions over zeros of general zeta and L -functions - [<http://arxiv.org/abs/1302.4644v1>] Heat kernels on regular graphs and generalized Ihara zeta function formulas - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/1009.0997v3>] Spectral asymptotics for Robin problems with a discontinuous coefficient - [<http://arxiv.org/abs/2103.14334v4>] Invariant subspaces of elliptic systems II: spectral theory - [<http://arxiv.org/abs/1906.02514v1>] Ihara Zeta Entropy - [<http://arxiv.org/abs/1809.06500v3>] Range entropy: A bridge between signal complexity and self-similarity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=180.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def is_connected(G):
    visited = [False] * len(G)
    stack = [0]
    while stack:
        u = stack.pop()
        if not visited[u]:
            visited[u] = True
```

```

        for v in range(len(G)):
            if G[u][v] and not visited[v]:
                stack.append(v)
    return all(visited)

def is_3_regular(G):
    return all(sum(G[u]) == 3 for u in range(len(G)))

def adjacency_matrix_to_laplacian(A):
    n = len(A)
    D = [sum(row) for row in A]
    L = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                L[i][j] = D[i]
            else:
                L[i][j] = -A[i][j]
    return L

def gaussian_elimination(A, b):
    n = len(A)
    M = [row[:] + [b[i]] for i, row in enumerate(A)]
    for i in range(n):
        max_row = max(range(i, n), key=lambda k: abs(M[k][i]))
        M[i], M[max_row] = M[max_row], M[i]
        for j in range(i + 1, n):
            factor = M[j][i] / M[i][i]
            for k in range(n + 1):
                M[j][k] -= factor * M[i][k]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (M[i][n] - sum(M[i][j] * x[j] for j in range(i + 1, n))) / M[i][i]
    return x

def solve_linear_system(A, b):
    n = len(A)
    if not is_square_matrix(A) or len(b) != n:
        raise ValueError("Invalid input dimensions")
    if not is_invertible(A):
        raise ValueError("Matrix is not invertible")
    return gaussian_elimination(A, b)

def max_decision_depth(T):
    def dpll(assignment, clause_index):
        if clause_index == len(T):
            return 0
        for var in range(len(T)):
            if var not in assignment:
                new_assignment = assignment.copy()
                new_assignment[var] = True
                if all(any(new_assignment[j] for j in clause) for clause in T[clause_index]):
                    depth_true = dpll(new_assignment, clause_index + 1)
                    if depth_true == float('inf'):
                        return float('inf')
                new_assignment[var] = False

```

```

        depth_false = dpll(new_assignment, clause_index + 1)
        if depth_false == float('inf'):
            return float('inf')
        return max(depth_true, depth_false) + 1
    return float('inf')
return dpll({}, 0)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [6, 8, 10, 12]
    results = []

    for n in n_values:
        G = [[0] * n for _ in range(n)]
        while not is_connected(G) or not is_3_regular(G):
            for u in range(n):
                neighbors = [v for v in range(n) if G[u][v]]
                if len(neighbors) < 2:
                    continue
                v, w = random.sample(neighbors, 2)
                G[u][v] = G[v][u] = 1
                G[u][w] = G[w][u] = 1

        A_G = [row[:] for row in G]
        λs = sorted(math.eigenvalsh(A_G), reverse=True)
        η_G = sum(math.log(abs(11 - 3 * λ)) for λ in λs) / n
        η_G /= len(G)

        for _ in range(5):
            c = [random.randint(0, 1) for _ in range(n)]
            if sum(c) % 2 != 1:
                continue

            T = []
            for u in range(n):
                for v in range(u + 1, n):
                    if G[u][v]:
                        T.append([u * 3 + i - 1 for i in range(4)])
                        T.append([v * 3 + i - 1 for i in range(4)])

            d_DPLL = max_decision_depth(T)
            results.append((n, η_G, d_DPLL))

    total_d_DPLL = sum(d_DPLL for _, _, d_DPLL in results)
    avg_d_DPLL = total_d_DPLL / len(results)
    std_d_DPLL = math.sqrt(sum((d_DPLL - avg_d_DPLL) ** 2 for _, _, d_DPLL in results) / len(results))

    support_fraction = sum(1 for n, η_G, d_DPLL in results if d_DPLL >= n * η_G / 4) / len(results)

    conjecture_holds = support_fraction >= 0.95
    counterexample = "" if conjecture_holds else "d_DPLL < |V| · η(G) / 4"

    return {
        "metric_name": "d_DPLL",
        "metric_value": avg_d_DPLL,

```

```

        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    avg_d_DPLL = sum(result["metric_value"] for result in results) / len(results)
    std_d_DPLL = math.sqrt(sum((result["metric_value"] - avg_d_DPLL) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={avg_d_DPLL} std={std_d_DPLL} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={first_failing_seed} d_DPLL < |V|·η(G)/4" first_failing_seed={first_failing_seed})
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 180s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 180s (returncode 124) without producing a RESULT line, so no empirical data is available to evaluate the pre-registered support condition. | next: Reduce instance scope (e.g., $|V| \in \{6, 8, 10\}$, ≤ 10 graphs each) or optimize DPLL depth computation and increase the timeout to obtain measurable data.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	96439
2	preregistration	claude_max	opus	0	0	7674
3	novelty	claude_max	opus	0	0	3100
4	novelty	claude_max	opus	0	0	8716
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19225
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17164
7	judge	claude_max	opus	0	0	4210

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 156527 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/3a9492e9db22.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3a9492e9db22.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3a9492e9db22.tar.gz` (if generated)